

Architecture du projet Onyx Messenger

Ce document détaille l'architecture technique du projet Onyx Messenger, expliquant les choix de conception et l'organisation du code.

Backend (Express.js avec Clean Architecture)

Le backend d'Onyx Messenger est construit selon les principes de la Clean Architecture, qui permet une séparation claire des responsabilités et facilite la maintenance et les tests.

Couches de l'architecture

1. **Domain / Entities** (Couche centrale)
 2. Contient les entités métier pures (User, Message, ConnectionLog)
 3. Indépendant de tout framework ou technologie
 4. Définit les règles métier fondamentales
5. **Application / Use Cases** (Couche intermédiaire)
 6. Contient la logique applicative
 7. Orchestration des flux de travail
 8. Dépend uniquement du Domain
 9. Définit les interfaces (ports) pour les repositories et services
10. **Infrastructure** (Couche externe)
 11. Implémentations concrètes des interfaces définies dans l'Application
 12. Intégration avec les technologies externes (Prisma, WebSockets, JWT)
 13. Adaptateurs pour les frameworks et bibliothèques
14. **Presentation / API** (Couche externe)
 15. Point d'entrée de l'application
 16. Contrôleurs et routes Express
 17. Validation des entrées et sérialisation des réponses

Flux de données

1. Les requêtes HTTP arrivent via les routes Express (Presentation)
2. Les contrôleurs valident les entrées et appellent les cas d'utilisation appropriés (Application)
3. Les cas d'utilisation orchestrent les opérations en utilisant les entités (Domain) et les interfaces de repository
4. Les implémentations concrètes des repositories (Infrastructure) interagissent avec la base de données
5. Les résultats remontent la chaîne jusqu'au client

Inversion de dépendance

Un principe clé de la Clean Architecture est l'inversion de dépendance : - Les couches internes ne dépendent jamais des couches externes - Les dépendances pointent vers l'intérieur - Les interfaces (ports) sont définies dans les couches internes - Les implémentations (adaptateurs) sont dans les couches externes

Frontend (Next.js avec Redux Toolkit)

Le frontend d'Onyx Messenger est construit avec Next.js et utilise Redux Toolkit pour la gestion de l'état.

Organisation du code

1. **App Router**
2. Structure basée sur le système de routage App Router de Next.js
3. Organisation des pages par fonctionnalité (/login, /register, /dashboard)
4. Utilisation des layouts pour le partage de composants entre les routes
5. **Components**
6. Organisation par domaine fonctionnel (auth, chat, layout, ui)
7. Composants réutilisables et modulaires
8. Séparation des préoccupations (présentation vs logique)
9. **State Management**
10. Redux Toolkit pour la gestion de l'état global
11. Slices organisés par domaine (auth, users, chat)
12. RTK Query pour les appels API et la gestion du cache

13. **WebSockets**

14. Hook personnalisé (useWebSocket) pour encapsuler la logique WebSocket
15. Intégration avec Redux pour mettre à jour l'état en fonction des événements WebSocket
16. Gestion de la reconnexion automatique

Flux de données

1. Les actions utilisateur déclenchent des actions Redux ou des appels API via RTK Query
2. Les reducers mettent à jour l'état global
3. Les composants se re-rendent en fonction des changements d'état
4. Les événements WebSocket sont traités et déclenchent des actions Redux
5. Les composants réagissent aux mises à jour en temps réel

Communication entre Frontend et Backend

1. **API REST**
2. Utilisée pour les opérations CRUD standard
3. Authentification via JWT
4. Endpoints organisés par ressource (/api/auth, /api/users, /api/messages)
5. **WebSockets**
6. Communication bidirectionnelle en temps réel
7. Utilisée pour les messages instantanés et les mises à jour de statut
8. Authentification via token dans l'URL de connexion

Base de données

Le schéma de base de données est conçu pour optimiser les requêtes fréquentes :

1. **Tables principales**
2. User: stocke les informations utilisateur et le statut
3. Message: stocke les messages échangés entre utilisateurs
4. ConnectionLog: enregistre les connexions et déconnexions
5. **Relations**

6. One-to-Many entre User et Message (un utilisateur peut envoyer/recevoir plusieurs messages)
7. One-to-Many entre User et ConnectionLog (un utilisateur peut avoir plusieurs journaux de connexion)
8. **Optimisations**
9. Index sur les colonnes fréquemment interrogées
10. Relations bien définies pour faciliter les jointures

Justification des choix techniques

1. **Clean Architecture (Backend)**
2. Séparation claire des responsabilités
3. Facilité de test (les cas d'utilisation peuvent être testés indépendamment)
4. Flexibilité pour changer de technologies (ex: passer de Prisma à un autre ORM)
5. **Next.js avec App Router (Frontend)**
6. Routage moderne et performant
7. Rendu côté serveur pour de meilleures performances
8. Organisation intuitive du code
9. **Redux Toolkit et RTK Query**
10. Gestion simplifiée de l'état global
11. Réduction du code boilerplate
12. Gestion efficace du cache pour les requêtes API
13. **WebSockets pour la communication en temps réel**
14. Communication bidirectionnelle instantanée
15. Faible latence pour les messages
16. Mises à jour en temps réel des statuts utilisateur
17. **Prisma ORM**
18. API typée pour une meilleure sécurité
19. Migrations automatisées
20. Excellent support TypeScript